

“Eu termino isso aqui e depois te chamo.”

Essa é praticamente a ideia de uma callback.

---

## O Conceito Principal

Uma callback é:

uma função passada para outra função

para ser executada depois apenas.

---

## Funções em JavaScript São Valores

Em JavaScript, funções não são especiais.

Elas podem ser:

- guardadas em variáveis
  - passadas como argumentos
  - retornadas por funções
  - armazenadas em objetos
- 

## Exemplo

```
function ola() {  
  console.log("Olá")  
}
```

A função existe normalmente.

---

Mas também podemos fazer:

```
const minhaFuncao = ola
```

Agora:

```
minhaFuncao()
```

executa:

```
Olá
```

---

## O Que Isso Significa?

Que funções podem viajar pelo programa como qualquer valor. De maneira livre do que e imaginado.

Assim:

```
function executar(funcao) {  
    funcao()  
}
```

---

## Passando uma função

```
function dizerOi() {  
    console.log("Oi, Pacoquero!")  
}  
  
executar(dizerOi)
```

---

## Fluxo Mental

1.

```
executar(dizerOi)
```

↓

A função `dizerOi` é enviada como argumento.

---

## 2.

Dentro de `executar` :

```
funcao()
```

↓

isso vira:

```
dizerOi()
```

---

## 3.

Executa:

```
Oi, Pacoquero!
```

---

## A Parte MAIS Importante

Existe uma diferença **GIGANTE** entre:

---

## Passar a função

```
executar(dizerOi)
```

Você está dizendo:

```
"toma essa função para usar depois"
```

---

## Executar a função

```
executar(dizerOi())
```

Aqui você executa IMEDIATAMENTE.

O resultado da função é enviado na hora. De forma síncrona, sem nenhum delay.

---

## Visualmente

### Sem parênteses

```
dizerOi
```

```
referência da função
```

---

### Com parênteses

```
dizerOi()
```

```
execução da função
```

---

## Por Que Callbacks Existem?

Porque algumas coisas demoram.

Exemplo:

- clique do usuário
- resposta da internet
- leitura de arquivo
- temporizador

O JavaScript não quer travar tudo esperando.

Então ele diz:

```
"continua o programa e depois eu te aviso"
```

---



## Exemplo com `setTimeout`

```
setTimeout(() => {  
  console.log("Executou")  
}, 3000)
```

---

## O Que o JavaScript Faz?

### 1.

Registra a callback.

```
"essa função deve rodar depois"
```

---

### 2.

Continua executando o resto do código.

---

### 3.

Após 3 segundos:

```
executa callback
```

---

## Visualmente

```
setTimeout()  
  ↓  
guarda callback  
  ↓  
espera 3 segundos  
  ↓  
executa função
```

---

## Callback em Eventos

Eventos são um dos usos mais importantes.

---

## Exemplo

```
button.addEventListener("click", () => {  
  console.log("clicou")  
})
```

---

## O Que Acontece Aqui?

### O navegador faz:

```
"quando clicar,  
execute essa função"
```

---

## A callback fica armazenada

Ela NÃO executa imediatamente.

Ela fica esperando o evento acontecer.

---

## Fluxo

```
usuário clica
    ↓
navegador detecta
    ↓
callback executa
```

---

## Callback Síncrona

Executa imediatamente.

---

## Exemplo

```
const numeros = [1, 2, 3]

numeros.forEach((numero) => {
  console.log(numero)
})
```

---

## O que acontece?

O `forEach` percorre o array.

Para cada item:

```
executa callback
```

---

## Internamente seria algo parecido com:

```
function forEach(callback) {
  callback(1)
  callback(2)
  callback(3)
}
```

---

# Callback Assíncrona

Executa depois.

---

## Exemplo

```
fetch("/usuarios")
```

A internet demora.

Então o JavaScript usa callbacks/promises para continuar funcionando sem travar.

---

## Controle de Sequência

Callbacks permitem decidir:

```
o que fazer depois
```

---

## Exemplo Sem Callback

```
function soma(a, b) {  
    return a + b  
}  
  
const resultado = soma(5, 5)  
  
mostrar(resultado)
```

---

## Exemplo Com Callback

```
function soma(a, b, callback) {  
    const resultado = a + b
```



```
    callback(resultado)
}
```

---

## Chamando

```
soma(5, 5, mostrar)
```

---

## O Que Isso Permite?

A função `soma` não precisa saber:

- como mostrar
- onde mostrar
- o que fazer depois

Ela apenas:

```
executa callback
```

---

## Isso Dá Flexibilidade

A mesma função pode fazer coisas diferentes.

---

## Exemplo

```
soma(5, 5, console.log)
```

ou:

```
soma(5, 5, alert)
```

---



# Callbacks Estão em Todo Lugar

| Recurso          | Callback |
|------------------|----------|
| addEventListener | ✓        |
| setTimeout       | ✓        |
| setInterval      | ✓        |
| fetch            | ✓        |
| forEach          | ✓        |
| map              | ✓        |
| filter           | ✓        |

---

## Problema Famoso: Callback Hell

Muitas callbacks aninhadas:

```
func1(() => {  
  func2(() => {  
    func3(() => {  
      func4()  
    })  
  })  
})
```

Isso vira bagunça.

---

## Por Isso Surgiram

- Promises
- async/await

Mas ambos ainda usam a ideia de callbacks internamente.

---

## Resumo Mental Correto

# Callback NÃO é algo mágico.

É apenas:

uma função entregue para outra função executar depois

---

## Resumo Final

### Callback

função passada como argumento

---

### Pode executar:

| Tipo       | Momento       |
|------------|---------------|
| síncrona   | imediatamente |
| assíncrona | depois        |

---

### Principal objetivo

controlar quando algo deve acontecer

---

### Exemplo Final Completo

```
function mostrar(valor) {  
  console.log(valor)  
}  
  
function calcular(a, b, callback) {  
  const soma = a + b
```

```
    callback(soma)
  }

  calcular(10, 5, mostrar)
```

---

## Fluxo

```
calcular()
  ↓
recebe callback
  ↓
faz cálculo
  ↓
executa callback
  ↓
mostrar(resultado)
```